

Section 02

Industrial Security (Bachelor)

Section 02 – ICS and SCADA Fundamentals



ICS and SCADA Fundamentals

Lecture Notes

Department of Computer Science

2026-05-17

Industrial Security (Bachelor)

Section 02

Industrial Security (Bachelor)



Section 02 – ICS and SCADA Fundamentals

ICS and SCADA Fundamentals
Lecture Notes
Department of Computer Science

- 1 Control Loop and Field Devices
- 2 Plant-Wide Architectures
- 3 Safety Layer

By the end of this section you will be able to

- Identify the components of a typical industrial control loop.
- Distinguish PLC, RTU, DCS, SCADA, HMI, Historian, and SIS.
- Explain the scan cycle and determinism requirements of PLCs.
- Read a simple ladder-logic program.
- Recognise why Safety Instrumented Systems must remain independent.

2026-05-17

Industrial Security (Bachelor)

Learning Objectives

Learning Objectives

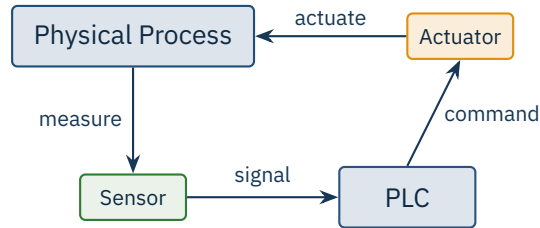
02

 By the end of this section you will be able to

- Identify the components of a typical industrial control loop.
- Distinguish PLC, RTU, DCS, SCADA, HMI, Historian, and SIS.
- Explain the scan cycle and determinism requirements of PLCs.
- Read a simple ladder-logic program.
- Recognise why Safety Instrumented Systems must remain independent.

1

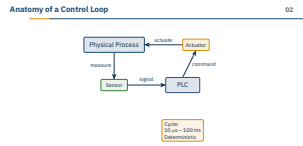
Control Loop and Field Devices



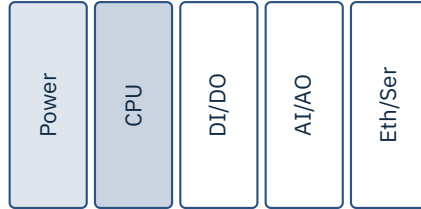
Cycle:
10 μ s – 100 ms
Deterministic

2026-05-17

Walk them through the loop physically: a thermocouple drops 4–20 mA into an analog input card, the PLC scans, the output card drives a contactor, the contactor opens a valve. Pre-empt the IT-brain misconception: in OT, input is *polled* every cycle, not event-driven — a missed sensor edge between scans is simply lost. A war story I tell: a Siemens S7-300 in a brewery whose cycle slowed from 8 to 60 ms after someone added a *single* chatty Modbus poll, and the dosing valve started overshooting. Ask the room: “where in this loop would *you* attack?” — they will say PLC, the right answer is usually the sensor or the engineering link. Transition: next slide opens the PLC box itself.



- Ruggedised industrial computer
- First model: Modicon 084 (1969)
- Replaced relay logic in plants
- IEC 61131-3 languages:
 - Ladder Diagram (LD)
 - Structured Text (ST)
 - Function Block (FBD)
 - Sequential Function Chart
 - Instruction List



Source: D. Morley, "History of the PLC", Modicon, 2003 | IEC 61131-3:2013.

2026-05-17

Industrial Security (Bachelor)

Control Loop and Field Devices

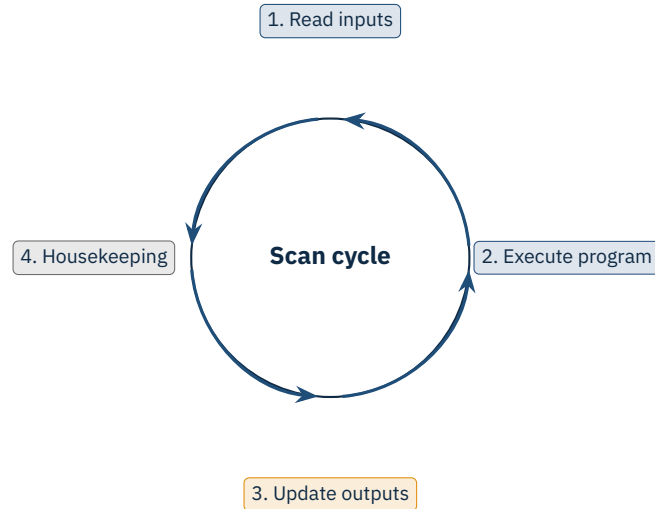
Programmable Logic Controllers (PLCs)

The Modicon 084 story is worth telling: GM wanted to stop rewiring relay panels at every model-year change, and Dick Morley's team shipped a box that was effectively a relay panel emulator — which is why ladder logic still looks like a wiring diagram fifty years later. Vendor quirks worth naming: Siemens uses STEP 7 / TIA Portal and a proprietary S7 protocol on port 102, Rockwell/Allen-Bradley uses Studio 5000 and EtherNet/IP (CIP), Beckhoff runs TwinCAT on a Windows PC with a real-time kernel extension. Pre-empt the misconception "PLCs are just embedded Linux now" — many still run bare-metal or a tiny RTOS exactly so the scan cycle stays predictable. A PLC will happily reboot itself if its watchdog detects the scan ran long — I have seen a packaging line trip because someone uploaded a too-large recipe block. Transition: now look at *how* that program runs, cycle by cycle.

Programmable Logic Controllers (PLCs)

- Ruggedised industrial computer
- First model: Modicon 084 (1969)
- Replaced relay logic in plants
- IEC 61131-3 languages:
 - Ladder Diagram (LD)
 - Structured Text (ST)
 - Function Block (FBD)
 - Sequential Function Chart
 - Instruction List

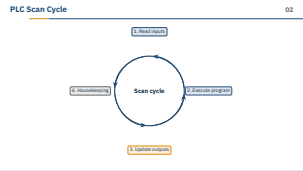
Source: D. Morley, "History of the PLC", Modicon, 2003 | IEC 61131-3:2013.



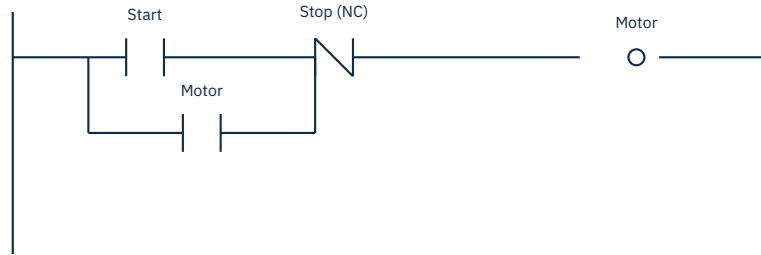
2026-05-17

Industrial Security (Bachelor)
└─ Control Loop and Field Devices

└─ PLC Scan Cycle



Make the scan cycle tangible: a typical Siemens S7-1500 or Rockwell ControlLogix runs a cycle in single-digit milliseconds, every cycle, forever — a PLC is essentially a `while (true)` that hates being interrupted. The big IT-side misconception to pre-empt is “PLCs are not real-time” — they usually are, with hard watchdog limits, which is exactly why a flood of network packets can crash one. Housekeeping (step 4) is where the comms stack and the engineering channel get serviced; that is where a noisy attacker pushes scan time over the watchdog. Ask the class: “what happens to cycle time if the PLC has to also serve a Modbus client every 10 ms?” That sets up jitter and determinism. Transition: next we look at what the *program* in step 2 actually looks like.



What it does

Start energises the Motor coil. The seal-in branch around Start latches the coil so it stays on. Stop (NC) breaks both paths.

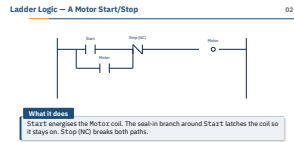
2026-05-17

Industrial Security (Bachelor)

Control Loop and Field Devices

Ladder Logic — A Motor Start/Stop

Read the rung out loud the way an electrician would: “power, Start (NO) closes, current goes through Stop (NC) which is closed by default, energises the Motor coil”. Emphasise why ladder logic survives despite looking ancient: a maintenance electrician with no software background can troubleshoot it on a printout with a multimeter — that is a feature, not a bug, and the reason Rockwell, Mitsubishi and Schneider all still ship LD-first toolchains. Pre-empt the misconception “Stop should be NO” — it is wired NC on purpose so a broken Stop wire *stops* the motor (fail-safe). The seal-in (latching) pattern is the single most common construct in industry; once they see it once they will see it everywhere. Optional question: “what if both Start and Stop are pressed at the same time?” (Stop wins, because it is in series.) Transition: zoom out from one rung to a whole plant.



What it does

Start energises the Motor coil. The seal-in branch around Start latches the coil so it stays on. Stop (NC) breaks both paths.

2

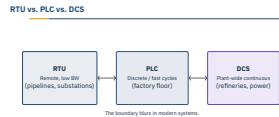
Plant-Wide Architectures



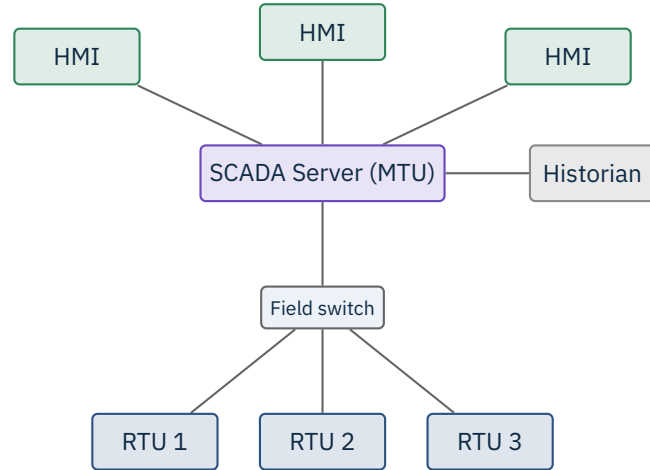
The boundary blurs in modern systems.

2026-05-17

Industrial Security (Bachelor)
 └ Plant-Wide Architectures
 └ RTU vs. PLC vs. DCS



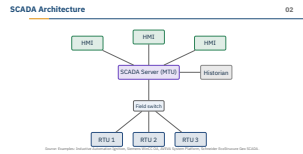
Anchor the distinction in geography and bandwidth: an RTU sits in a substation or pipeline shed and traditionally talked over a 1200 baud radio link with DNP3 (the protocol was literally designed for unreliable telemetry); a PLC sits on a factory floor on a switched Ethernet ring; a DCS (Emerson DeltaV, Yokogawa Centum, ABB 800xA) is a vendor-integrated *system* of controllers, HMIs and engineering tools sold as one piece. The misconception to preempt: students think these are three different boxes — in 2026 a Schneider M580 “PLC” can act as an RTU and be a node in an EcoStruxure DCS. The boundary really does blur. Optional question: “why do oil and gas pipelines still favour DNP3 over a richer protocol?” — answer: it tolerates dropouts and was designed for store-and-forward over flaky links. Transition: next slide pulls the supervisory layer on top of all three.



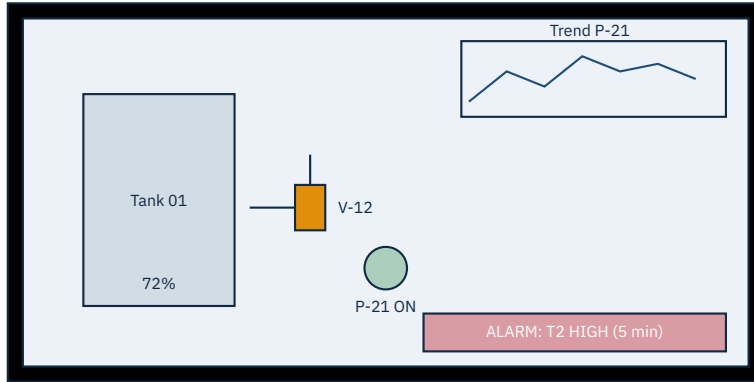
Source: Examples: Inductive Automation Ignition, Siemens WinCC OA, AVEVA System Platform, Schneider EcoStruxure Geo SCADA.

2026-05-17

Industrial Security (Bachelor)
 └ Plant-Wide Architectures
 └ SCADA Architecture



Stress that SCADA is a *role*, not a product — it polls/aggregates, drives HMIs, and feeds the historian. The MTU (Master Terminal Unit) is the polling head; RTUs in the field answer. Pre-empt the misconception “SCADA controls the process” — no, the PLC/RTU controls; SCADA *supervises*. A war story: a water utility I once visited polled 400 RTUs over a single VSAT link; when the satellite rain-faded, the SCADA screens went red but the RTUs kept the pumps running for hours — that local autonomy is the point. Name the big four vendors on the source line so they can recognise screenshots later. Transition: next slide zooms into the HMI surface that operators actually stare at.

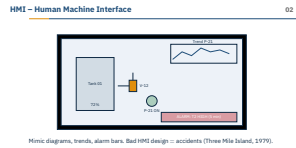


Mimic diagrams, trends, alarm bars. Bad HMI design = accidents (Three Mile Island, 1979).

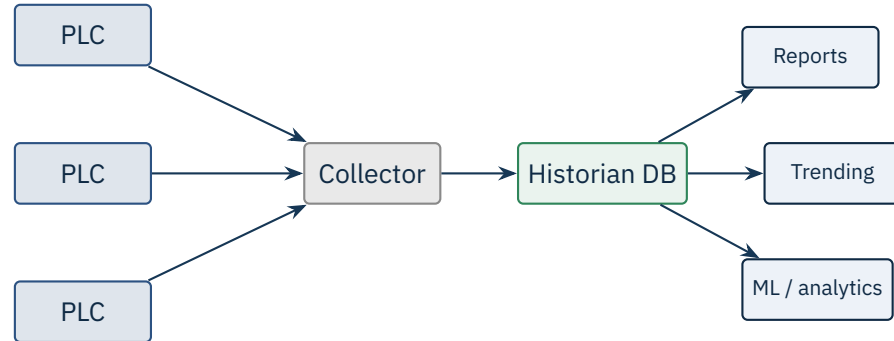
2026-05-17

Industrial Security (Bachelor)
└ Plant-Wide Architectures

└ HMI – Human Machine Interface



Three Mile Island is the canonical bad-HMI story: hundreds of alarms went off simultaneously and the operators could not see which one mattered — ISA-101 was written so this would not happen again. Modern guidance (also from the High Performance HMI movement) is greyscale by default and colour reserved for abnormal state, which is the opposite of what students expect from web UI. Pre-empt the misconception “the HMI is the control system” — it is just a window; pull the cable and the PLC keeps running. War story worth telling: Stuxnet specifically replayed normal-looking values to the WinCC HMI while spinning the centrifuges to destruction — operators saw green screens. Optional question: “which colour should a normally-open valve be?” — ISA-101 says muted grey, not green. Transition: where do those trend lines come from? The historian, next.

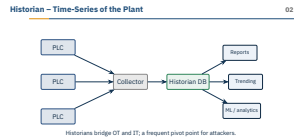


Historians bridge OT and IT; a frequent pivot point for attackers.

2026-05-17

Industrial Security (Bachelor)
└ Plant-Wide Architectures

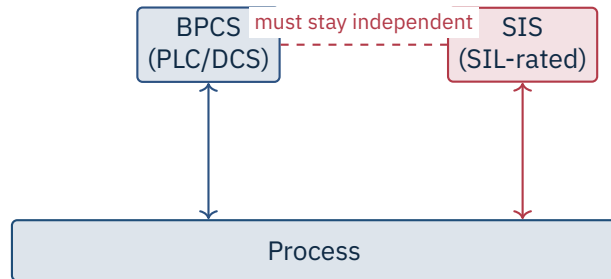
└ Historian – Time-Series of the Plant



Name the big products so they stick: OSIsoft PI (now AVEVA PI), GE Proficy, Aspen IP.21, Inductive Automation Ignition’s tag historian. These speak OT protocols downstream (OPC UA, S7, EtherNet/IP) and SQL/REST upstream to the IT business systems — which is exactly why attackers love them as a pivot. Pre-empt the misconception “it is just a database” — historians use compression (swinging-door, deadband) optimised for slowly-changing sensor values, so a generic Postgres dump will be 10–100× larger. War story: at one chemical plant the historian was the *only* machine dual-homed between the level-3 supervisory net and the level-4 business net, and the firewall rule was “permit any from historian” — classic. Optional question: “what does an attacker gain from read-only access to a historian?” — recipe IP, plant fingerprinting, alarm patterns. Transition: now to the safety layer that sits beside, not above, all of this.

3

Safety Layer



Safety layer

The SIS brings the process to a safe state on demand. **IEC 61511 / 61508**, SIL 1–4.

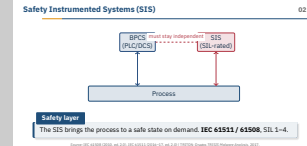
Source: IEC 61508 (2010, ed. 2.0), IEC 61511 (2016–17, ed. 2.0) | TRITON: Dragos TRISIS Malware Analysis, 2017.

2026-05-17

Industrial Security (Bachelor)

└ Safety Layer

└ Safety Instrumented Systems (SIS)



This is the most-misunderstood slide in the deck for IT-trained students — hammer it: SIS is *not* just another PLC. It is a separately certified box (Schneider Triconex, Siemens HIMA, Rockwell ICS Triplex, Emerson DeltaV SIS), often with triple-modular-redundant CPUs voting 2-out-of-3, and it must remain independent from the Basic Process Control System per IEC 61511. The dashed line in the diagram is the key visual: shared anything — engineering workstation, network segment, even time source — erodes the independence. War story is TRITON (2017), which compromised a Triconex specifically to disable safety so a later physical attack would not trip the shutdown; the malware was discovered only because it accidentally tripped the SIS into a safe state. Pre-empt: “why not just put safety logic in the DCS?” — because then a single common-mode bug or attack defeats both layers. Transition: back to the programming languages that make all of these controllers tick.



The big two in the wild

LD dominates discrete manufacturing; ST is the cross-vendor portable choice for new development.

2026-05-17

Industrial Security (Bachelor)
└ Safety Layer

└ IEC 61131-3 – PLC Programming Languages

IEC 61131-3 standardises the five languages but *not* the toolchain, so a Rockwell .ACD file does not open in TIA Portal and never will — vendor lock-in is by design. In the wild you mostly meet two: ladder for discrete manufacturing (because electricians can read it) and Structured Text for anything math-heavy or motion-control. FBD is the DCS world’s favourite (Emerson DeltaV, ABB 800xA). Instruction List is officially deprecated in the 2013 edition — mention only as a historical curiosity. Pre-empt the misconception “ST is just Pascal” — syntactically yes, semantically no: it runs inside the scan cycle, so there are no blocking calls, no malloc, no recursion in practice. Optional question: “why is recursion a bad idea in a PLC program?” — unbounded stack growth versus a fixed cycle budget. Transition: who writes these programs and from which machine — the engineering workstation.



Caution

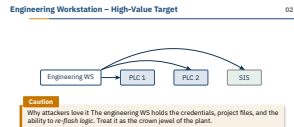
Why attackers love it The engineering WS holds the credentials, project files, and the ability to *re-flash logic*. Treat it as the crown jewel of the plant.

2026-05-17

Industrial Security (Bachelor)

└ Safety Layer

└ Engineering Workstation – High-Value Target



An engineering workstation is almost always a Windows box running TIA Portal, Studio 5000, EcoStruxure Control Expert or TwinCAT — and it usually carries the only copy of the master project file plus the cached PLC password. Compromise it and you can download arbitrary logic into the PLC, including the SIS if the network was not actually segmented. War story to tell: Stuxnet specifically targeted STEP 7 engineering workstations and injected its payload via the s7otbxdx.dll wrapper — it never touched the PLC over the wire directly, it waited for an engineer to plug in. Pre-empt the misconception “it is just a laptop, we can re-image it” — you cannot re-image the trust the PLC places in it, and you may not even have the original project file outside that laptop. Optional question: “where should the engineering WS live in the Purdue model?” — level 2, never roaming, never dual-homed. Transition: zoom out one more time to what kind of plant we are protecting.

2026-05-17

Process Types – Batch, Continuous, Discrete		
Type	Examples	Control system flavour
Continuous	Refinery, power, water	DCS, tight feedback control
Batch	Pharma, food, chemicals	DCS + recipe management (ISA-88)
Discrete	Auto assembly, packaging	PLCs, fast cycle, ladder logic

Why this matters for security

Different processes have different "safe states" – shutdown is fine for discrete, may be catastrophic for continuous. IR plans must reflect that.

Type	Examples	Control system flavour
Continuous	Refinery, power, water	DCS, tight feedback control
Batch	Pharma, food, chemicals	DCS + recipe management (ISA-88)
Discrete	Auto assembly, packaging	PLCs, fast cycle, ladder logic

Why this matters for security

Different processes have different “safe states” – shutdown is fine for discrete, may be catastrophic for continuous. IR plans must reflect that.

The takeaway here is that “just shut it down” is bad incident-response advice in OT. A discrete car-body line can stop mid-rung and resume; a continuous catalytic cracker cannot – cooling without flow cokes the tubes and turns a million-euro vessel into scrap. Batch plants (ISA-88 recipe management) sit in between: a hold-point exists, but only at specific phase transitions. Pre-empt the misconception “isolating the network is always safe” – yanking the network on a continuous DCS can starve the controllers of setpoints they need, which itself trips the plant. Optional question: “what is the safe state of an offshore gas platform?” – usually depressurise-and-vent, which is itself a controlled, hours-long procedure. Transition: wrap up with the summary takeaways for this section.

★ Key Takeaway: What to remember from this section

- Control loops close from sensors through PLC/DCS to actuators.
- SCADA aggregates field data; HMIs deliver situational awareness.
- Historians are the long-term memory and a common IT/OT bridge.
- SIS must remain independent — attacking it was TRITON's goal in 2017.

Use this slide as a verbal recap, not new content — ask them to name the four roles (controller, supervisor, archive, safety) before you click the bullets. The one sentence I want them to leave with: “SCADA *watches*, the PLC/DCS *controls*, the SIS *stops*, and the historian *remembers*.” Pre-empt the misconception that we have now covered networks — we have not; protocols like Modbus, OPC UA, EtherNet/IP and Profinet are still ahead. Optional sanity check: “which of these four would you compromise if your goal was a coordinated physical attack?” — TRITON answered: the SIS, exactly so the attack would not be stopped. Transition: Section 03 takes us into the OT network protocols themselves.

- IEC 61131-3:2013. *Programmable controllers – Part 3: Programming languages*.
- IEC 61508 (ed. 2.0, 2010). *Functional safety of E/E/PE safety-related systems*.
- IEC 61511 (ed. 2.0, 2016/17). *Functional safety – SIS for the process industry sector*.
- Morley, D. *The history of the PLC*. Modicon, 2003 (interview/archive).
- Dragos. *TRISIS Malware Analysis of Safety System Targeted Malware*, Dec 2017.
- ISA. *ISA-101 Human Machine Interfaces for Process Automation Systems*, 2015.
- ISA-88. *Batch Control Standards*.
- Krutz, R.L. *Securing SCADA Systems*, Wiley, 2006.
- Macaulay, T., Singer, B. *Cybersecurity for Industrial Control Systems*, CRC Press, 2012.

2026-05-17

Industrial Security (Bachelor)
└ Safety Layer

└ References – Section 02

- IEC 61131-3:2013. *Programmable controllers – Part 3: Programming languages*.
- IEC 61508 (ed. 2.0, 2010). *Functional safety of E/E/PE safety-related systems*.
- IEC 61511 (ed. 2.0, 2016/17). *Functional safety – SIS for the process industry sector*.
- Morley, D. *The History of the PLC*. Modicon, 2003 (interview/archive).
- Dragos. *TRISIS Malware Analysis of Safety System Targeted Malware*, Dec 2017.
- ISA. *ISA-101 Human Machine Interfaces for Process Automation Systems*, 2015.
- ISA-88. *Batch Control Standards*.
- Krutz, R.L. *Securing SCADA Systems*, Wiley, 2006.
- Macaulay, T., Singer, B. *Cybersecurity for Industrial Control Systems*, CRC Press, 2012.

End of Section 02

ICS and SCADA Fundamentals

Questions and discussion



2026-05-17

Industrial Security (Bachelor)
└─ Safety Layer

End of Section 02

ICS and SCADA Fundamentals

Questions and discussion

